

COMPSCI 701:
Creating Maintainable Software
Lecture 8.1: More Design Patterns

Ewan Tempero, Yu-Cheng Tu
School of Computer Science

Agenda

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

- Admin
 - ?
- More Design Patterns

More Design Patterns

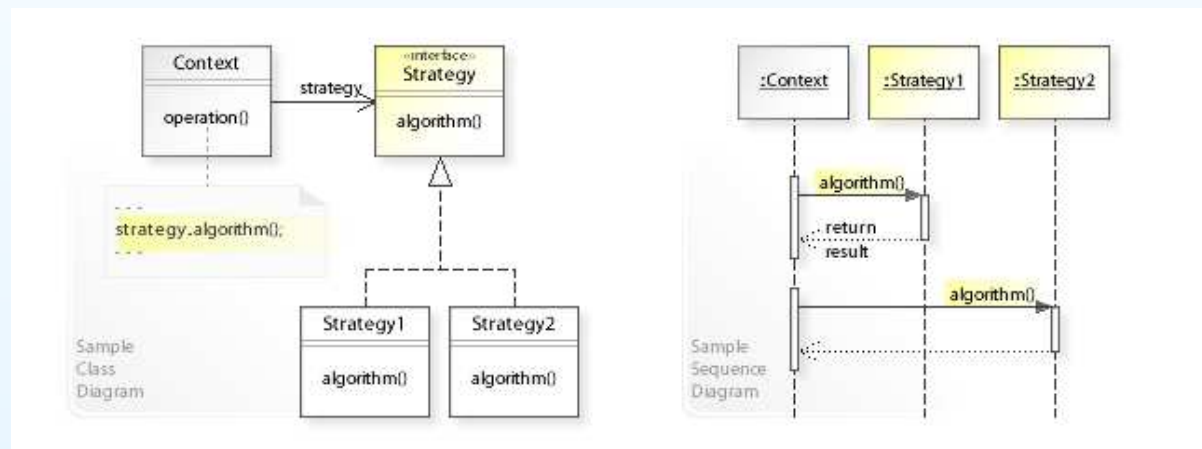
- Agenda
- **More Design Patterns**
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

- **Composite** — Compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly
- **Command** — Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations
- **Strategy** — Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.
- **Observer** — Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Decorator** — Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.
- **Factory Method** — Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- **Template Method** — Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.

Strategy

- Agenda
- More Design Patterns
- **Strategy**
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

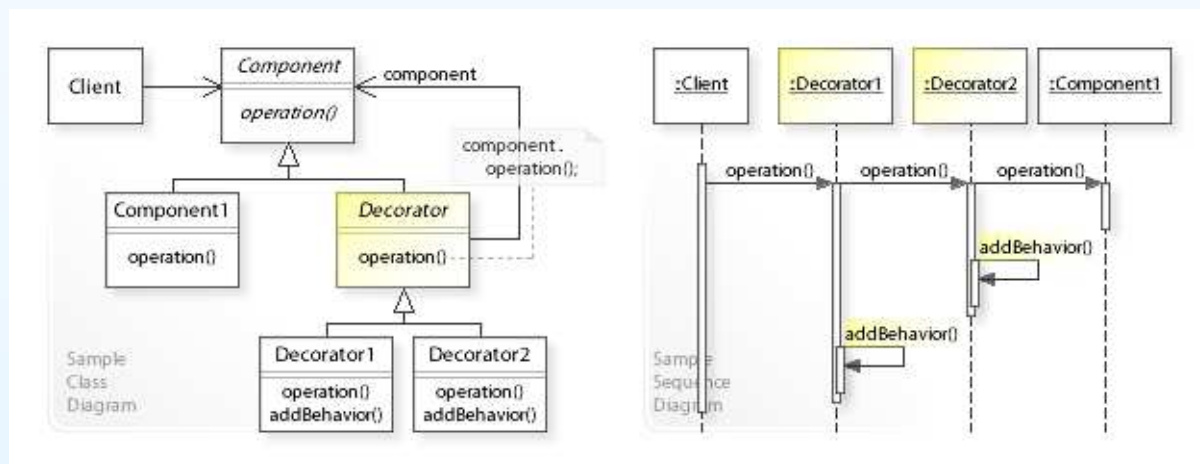
- Intent “Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”
- The Context maintains a reference to a Strategy object. The responsibility for performing an algorithm is delegated to a Strategy object.
- The Context is independent of how the algorithm is implemented.
- The Strategy defines a common interface for performing an algorithm.
- The algorithm or behaviour of an object could be changed at run-time.



Decorator

- Agenda
- More Design Patterns
- Strategy
- **Decorator**
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

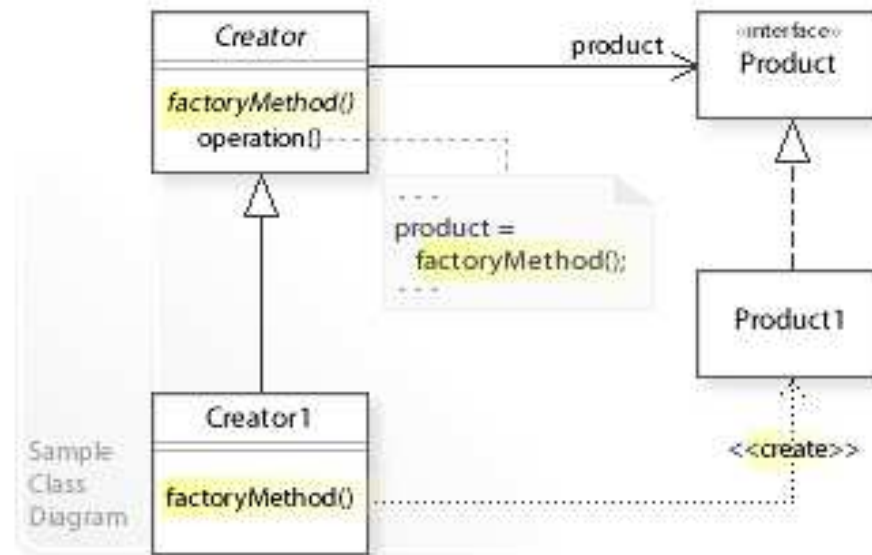
- Intent “Attach additional responsibilities to an object dynamically. Decorators provide a **flexible alternative to subclassing** for extending functionality”
- If you have an object and several responsibilities all of which can be added or removed independently, using inheritance will require a number of subclasses that is exponential in the number of responsibilities.
- A decorated concrete component is a component, and consists of the concrete component whose behaviour is modified by the decorator



Factory Method

- Agenda
- More Design Patterns
- Strategy
- Decorator
- **Factory Method**
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

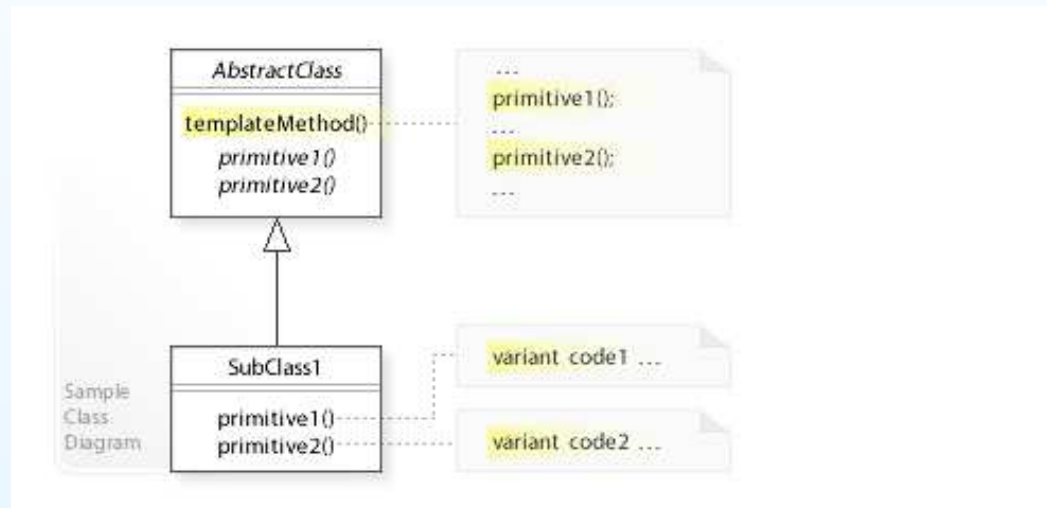
- Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
- Frameworks are built with abstractions, but must be made concrete somewhere — how to separate and yet control which concrete instances are created when
- A concrete Product is created by a concrete factory



Template Method

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- **Template Method**
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

- Intent “Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm’s structure.”
- The skeleton or “template” method is defined in a class with some steps deferred to (abstract) methods that must be defined in descendent classes



- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- **Template Method**
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Example

```
public abstract class BulkItem {  
    ...  
    public String getDescription() {  
        return getSpecifics() + " in bulk";  
    }  
    protected abstract String getSpecifics();  
    ...  
}
```

```
public class RainbowSweetFizzies extends BulkItem {  
    protected String getSpecifics() {  
        return "Rainbow Sweet Fizzies";  
    }  
}
```


Creating User Interfaces

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

- A user interface needs to show something about the underlying state of the system
- When the state of the system changes, the user interface may need to change to reflect the change in state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
- Some elements may have actions associated with them, but different elements will have different actions
- All of this is independent of what the system actually does

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
- When the state of the system changes, the user interface may need to change to reflect the change in state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
- Some elements may have actions associated with them, but different elements will have different actions

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
What shows the state is an **observer** of the state
- When the state of the system changes, the user interface may need to change to reflect the change in state
What shows the state is an **observer** of the state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
- Some elements may have actions associated with them, but different elements will have different actions

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- **MVC**
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
What shows the state is an **observer** of the state
- When the state of the system changes, the user interface may need to change to reflect the change in state
What shows the state is an **observer** of the state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
The elements are the *parts* of the user interface and groups of elements that behave like the whole user interface are **composites**
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
- Some elements may have actions associated with them, but different elements will have different actions

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- **MVC**
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
What shows the state is an **observer** of the state
- When the state of the system changes, the user interface may need to change to reflect the change in state
What shows the state is an **observer** of the state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
The elements are the *parts* of the user interface and groups of elements that behave like the whole user interface are **composites**
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
The associated elements **decorate** the other element
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
- Some elements may have actions associated with them, but different elements will have different actions

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- **MVC**
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
What shows the state is an **observer** of the state
- When the state of the system changes, the user interface may need to change to reflect the change in state
What shows the state is an **observer** of the state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
The elements are the *parts* of the user interface and groups of elements that behave like the whole user interface are **composites**
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
The associated elements **decorate** the other element
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
The user interface provides different **strategies** for how actions are performed
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
The user interface provides different **strategies** for how actions are performed
- Some elements may have actions associated with them, but different elements will have different actions

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- **MVC**
- MVC Alterability
- JUnit
- Class exercise
- Key Points

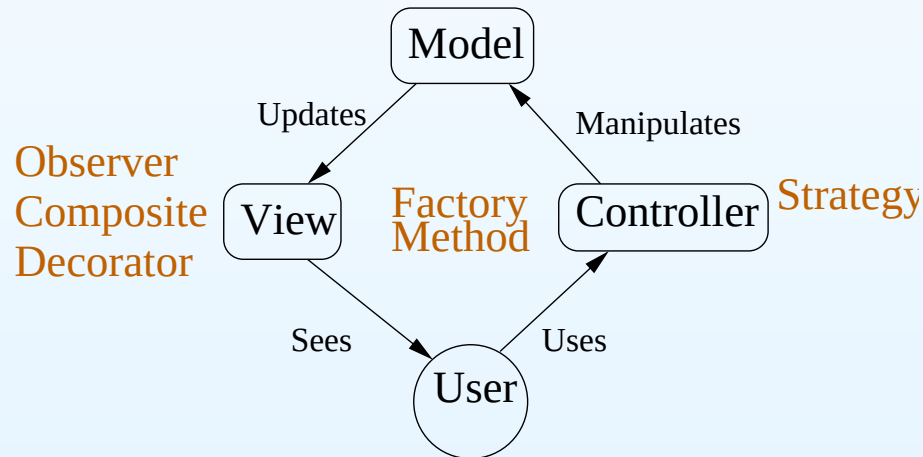
Creating User Interfaces

- A user interface needs to show something about the underlying state of the system
What shows the state is an **observer** of the state
- When the state of the system changes, the user interface may need to change to reflect the change in state
What shows the state is an **observer** of the state
- A user interface typically has multiple elements to it. Some elements must behave the same way as the whole user interface (e.g. resizing)
The elements are the *parts* of the user interface and groups of elements that behave like the whole user interface are **composites**
- Some elements of the interface must always be associated with some other element (e.g. scrollbars), and modifies some aspect of that element in some way
The associated elements **decorate** the other element
- The user interface has to provide the means for the user to interact (perform actions with) the user interface
The user interface provides different **strategies** for how actions are performed
- Exactly how a user performs a specific action may differ e.g. from a menu, or with a keyboard short-cut
The user interface provides different **strategies** for how actions are performed
- Some elements may have actions associated with them, but different elements will have different actions
The decision for what the actions are for the elements can be deferred and constructed through a **factory method**

Model-View-Controller (MVC)

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

- The “original” pattern (pre 1980), but any implementation usually involves other patterns
- Provides solutions to several problems associated with developing user interfaces
 - Model — the state of the system that the user wants to see some representation of and wants to be able to manipulate in some way
 - View — a representation of the Model presented to the user
 - Controller — the means with which the User performs actions to manipulate the model
- Like other patterns, has a number of variants when implemented



Changing Systems with User Interfaces

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- **MVC Alterability**
- JUnit
- Class exercise
- Key Points

- Change what state of the system has to be shown
- Change how the state of system is shown
- Change how the user performs actions in the system

MVC and Alterability

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- **MVC Alterability**
- JUnit
- Class exercise
- Key Points

- MVC allows a number of change cases to be implemented with only the minimum number of existing classes needing to change (*C*)
- Often new functionality is added in new classes, not by changing existing classes
- Change what state of the system has to be shown

Change the model (if need be), change *only* the views affected (which possibly only need to be added), usually *no need to change* any controllers
- Change how the state of system is shown
- Change how the user performs actions in the system

MVC and Alterability

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- **MVC Alterability**
- JUnit
- Class exercise
- Key Points

- MVC allows a number of change cases to be implemented with only the minimum number of existing classes needing to change (*C*)
- Often new functionality is added in new classes, not by changing existing classes
- Change what state of the system has to be shown

Change the model (if need be), change *only* the views affected (which possibly only need to be added), usually *no need to change* any controllers
- Change how the state of system is shown

Change the relevant view, usually *no need to change* model or any controllers
- Change how the user performs actions in the system

MVC and Alterability

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- **MVC Alterability**
- JUnit
- Class exercise
- Key Points

- MVC allows a number of change cases to be implemented with only the minimum number of existing classes needing to change (*C*)
- Often new functionality is added in new classes, not by changing existing classes
- Change what state of the system has to be shown

Change the model (if need be), change *only* the views affected (which possibly only need to be added), usually *no need to change* any controllers
- Change how the state of system is shown

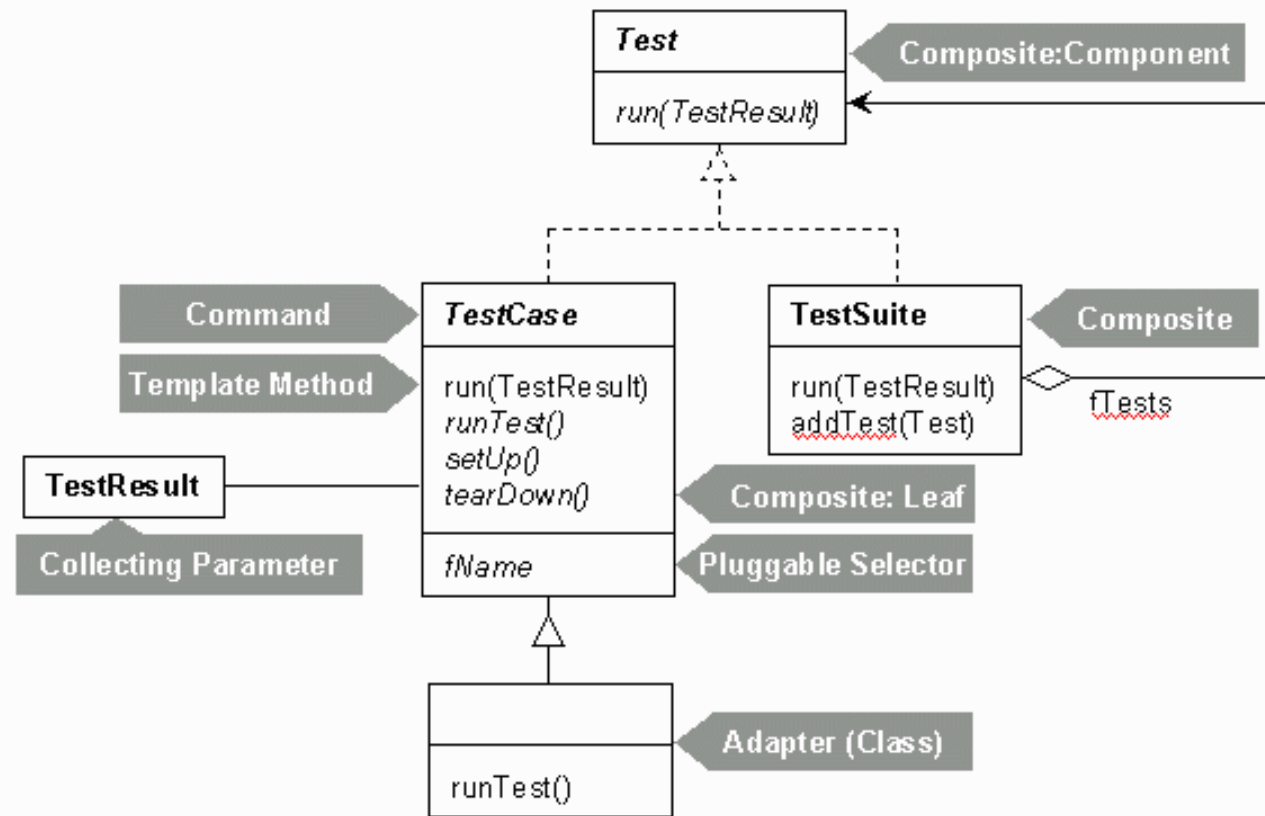
Change the relevant view, usually *no need to change* model or any controllers
- Change how the user performs actions in the system

Change (or add) *only* the relevant controllers usually *no need to change* any controllers

JUnit Example

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- **JUnit**
- Class exercise
- Key Points

- JUnit uses a number of patterns



(<https://curlunit.sourceforge.net/doc/cookstour/cookstour.htm>)

Use of Template Method

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

```
public abstract class TestCase extends Assert implements Test {
    public void runBare() throws Throwable {
        Throwable exception = null;
        setUp();
        try {
            runTest();
        } catch (Throwable running) {
            exception = running;
        } finally {
            try {
                tearDown();
            } catch (Throwable tearingDown) {
                if (exception == null) exception = tearingDown;
            }
        }
        if (exception != null) throw exception;
    }
    ...
    protected void setUp() throws Exception

    protected void tearDown() throws Exception
}
```

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Class exercise

- Discuss how use of Design Patterns generally impacts maintainability,
 - Give an example using one of the Design Patterns discussed in class that could help maintainability.
 - Given an example using one of the Design Patterns discussed in class that could hinder maintainability.

- Agenda
- More Design Patterns
- Strategy
- Decorator
- Factory Method
- Template Method
- MVC
- MVC Alterability
- JUnit
- Class exercise
- Key Points

Key Points

- Design Patterns often do not appear by themselves, there are often other patterns close by
- Most design patterns support Alterability by allowing new functionality to be supplied in new classes, and by reducing the number of existing classes that need to change